

Artificial Intelligence and Knowledge Engineering

Assignment 5

Paweł Walkowiak

June 3, 2026

1 Objective of the Assignment

The objective of this assignment is to familiarize you with the capabilities of language models based on a classic task, which is text classification.

2 Theoretical Introduction

2.1 Text Classification

Text classification is one of the fundamental tasks of Natural Language Processing (NLP). It consists in automatically assigning a label (class) $c \in C$ to an unstructured text t using a classifier f . From a mathematical point of view, this process can be represented as a function: $f(t) \rightarrow c$. In the case of sentiment analysis, the set of categories C usually includes values such as **plus** (positive), **minus** (negative), **neutral** (neutral), and **ambiguous** (mixed). The modern approach relies on representing text in the form of dense vectors of real numbers, which encode the semantic meaning of the utterance.

2.2 Large Language Models (LLMs)

Large Language Models (LLMs) are models based on the Transformer architecture, possessing billions of parameters and trained on gigantic datasets. Most contemporary models (e.g., GPT-4, Llama 3, Phi-2) are **generative Decoder-only models**.

- **Principle of Operation:** These models predict the next most probable token (a word or a word fragment) based on the provided context.
- **Application in Classification:** LLMs do not require training for a specific task (known as *Zero-shot learning*). By appropriately formulating instructions (*Prompt Engineering*), we force the model to generate the name of a category instead of a free-form response.

2.3 Encoder-only Models (e.g., BERT)

Before the era of generative LLMs, the standard was **Encoder-only models**, whose most popular representative is BERT (*Bidirectional Encoder Representations from Transformers*).

- **Specialization:** These models do not generate text smoothly; instead, they transform text into a vector representing the entire sentence (often associated with a special token [CLS]), which allows for classification by adding an appropriate classification head.
- **Performance:** They are significantly smaller and faster than generative models, making them ideal for mass classification tasks where speed and computational cost are important.

2.4 Working Environment: Google Colab

In order to provide the necessary computing power required to run language models, it is recommended to use the **Google Colab** environment. It is a free cloud-based service that allows you to write and execute Python code in your browser, offering free access to hardware accelerators like **GPUs** (e.g., NVIDIA T4). To take advantage of the graphics card, which will significantly speed up the text generation process (inference), you should change the runtime type after launching the notebook:

1. Click on the **Runtime** menu.
2. Select the **Change runtime type** option.
3. In the **Hardware accelerator** section, select **T4 GPU**.
4. Confirm with the **Save** button.

Using a GPU allows the model to be loaded in **bfloat16** or **float16** format, which reduces VRAM consumption and shortens the waiting time for the model's response.

3 Dataset Description: PolEmo2.0-IN

As part of this task, we will address text sentiment classification (*Sentiment Analysis*) using the **allegro/klej-polemo2-in** dataset. This is one of the key components of the KLEJ benchmark (*Comprehensive Polish Language Evaluation*), which is used to evaluate the capabilities of language models for the Polish language.

3.1 Characteristics of the Dataset

The PolEmo2.0-IN dataset contains online reviews from various domains (e.g., medicine, hotels). The task is to assign one of four emotional categories to a given review.

- **Source:** User reviews from websites.
- **Domain:** Reviews from the fields of medicine and hospitality (*in-domain* dataset).
- **Number of Classes:** 4 sentiment categories:
 - **plus** – unambiguously positive opinion,
 - **minus** – unambiguously negative opinion,
 - **neutral** – descriptive opinion, devoid of emotional charge,
 - **ambiguous** – opinion containing both positive and negative elements (mixed).

3.2 Data Structure

The loaded dataset has the following fields:

sentence: The content of the review in Polish (input data for the model).

target: The class identifier (`__label__meta_minus_m`, `__label__meta_plus_m`, `__label__meta_zero`, `__label__meta_amb`).

3.3 Dataset Loading Instructions

To load the data in a Python environment, use the **datasets** library from Hugging Face:

```
from datasets import load_dataset

dataset = load_dataset("allegro/klej-polemo2-in", split="test")

print(f"Text:{dataset[0]['sentence']}")
print(f"Label:{dataset[0]['target']}")
```

Listing 1: Loading the PolEmo2.0 dataset

4 Model Loading Examples

4.1 Loading an encoder-only model:

```
from transformers import pipeline
import torch

encoder_model_name = "Voicelab/herbert-base-cased-sentiment"
sentiment_pipeline = pipeline("text-classification", model=encoder_model_name,
                              device=0 if torch.cuda.is_available() else -1)

sample_text = "Ala ma kota"
prediction = sentiment_pipeline(sample_text)
print(f"Prediction: {prediction}")
```

Listing 2: Loading a model using transformers.pipeline

4.2 Loading an LLM:

```
import torch
from transformers import pipeline, AutoModelForCausalLM, AutoTokenizer
from langchain_huggingface import HuggingFacePipeline
from langchain_core.prompts import PromptTemplate

llm_model_name = "Qwen/Qwen2.5-1.5B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(llm_model_name)
model = AutoModelForCausalLM.from_pretrained(
    llm_model_name,
    torch_dtype=torch.float16,
    device_map="auto"
)

hf_pipeline = pipeline("text-generation", model=model, tokenizer=tokenizer,
                      temperature=0.1, do_sample=True, pad_token_id=tokenizer.eos_token_id)
llm = HuggingFacePipeline(pipeline=hf_pipeline)

basic_template = """Classify the text sentiment into one of three classes:
    positive, negative, neutral.
Text: {text}
Class: """

prompt = PromptTemplate.from_template(basic_template)
llm_chain = prompt | llm

result = llm_chain.invoke({"text": sample_text})
print(result)
```

Listing 3: Loading an LLM

5 Task

Your task is to develop a comparison of text classification on the given dataset using both encoder and decoder model approaches. In each case, you must report the results using standard metrics such as Accuracy, F1, and pay attention to the classification quality for individual classes. Particular attention should be paid to mapping the text labels (generated by the model) into numerical identifiers consistent with the structure of the dataset.

Scoring:

1. Load the PolEmo2.0-IN dataset and check basic statistics of the dataset (number of samples in subsets, class balance, text length). NOTE: for the purpose of this assignment, use only examples from the test split without the `ambiguous` class. (10 points)
2. Perform classification using a pretrained encoder-only model. Example model recommended to start with: `Voicelab/herbert-base-cased-sentiment`. (20 points)
3. Explore parameters to improve/compare classification results (temperature, alternative classification model). Other models can be found on https://huggingface.co/models?pipeline_tag=text-classification (20 points)
4. Perform classification using a decoder-only LLM (the size of the model should be chosen according to the available computational resources). Model recommended to start with: `Qwen/Qwen2.5-1.5B-Instruct` (20 points)
5. Explore parameters to improve classification results (prompt, temperature, quantization, parsing results using e.g., `langchain_core.output_parsers.JsonOutputParser`). (30 points)

The parameter exploration should investigate at least two aspects, for example: how the generation temperature affects the results and how the base model affects the classification results. Please do not worry about using smaller models (such as 1.5B/3B) instead of larger ones; the goal of this assignment is to build intuition in using LLMs, which will be useful in future projects.

Prepare a report for this assignment containing a brief description of all the steps performed, along with the results of the task and their interpretation. In the report, indicate the source materials used and briefly describe the libraries used in the implementation. Submit the report to the instructor at least 24 hours before the assignment deadline.

6 Appendix - Useful Libraries for Python

- `import torch`
- `from transformers import pipeline`
- `from datasets import load_dataset`
- `from langchain_huggingface import HuggingFacePipeline`
- `from langchain_core.prompts import PromptTemplate`
- `from langchain_core.output_parsers import JsonOutputParser`
- `from pydantic import BaseModel, Field`